

Session 3 — Homework

Bringing two sets of data together

Hand in before: the agreed date at the end of the course **Time you need:** about 60–90 minutes for CORE

Objective

In class the API gave us a list of tasks and we put it on the screen.

This homework goes one step further: **one request is not always enough**. You will ask the API a second question, and then make the two answers work together.

Everything you need is something you have already used: `fetch`, `async` / `await`, `try` / `catch`, `for...of`, `if`, functions and template literals. Nothing new has to be learned — the new part is **deciding what data you need** and how to connect two lists.

What you already have

Open your project and check that all of this still works. It is **not** homework — it is the application you are building on:

- tasks arriving from the API when the page opens,
- the three statistic cards (Total, Completed, Pending),
- the **All / Completed / Pending** buttons,
- the **search box** that filters by title while you type,
- search and the status buttons working **together**,
- the progress sentence under the list,
- the "Loading tasks..." message and the red error message.

Nothing on that list is part of this homework. All of it must still work when you are finished.

Look at your data first

Open this address in a browser tab and read one task carefully:

```
https://jsonplaceholder.typicode.com/todos?_limit=15
```

You will see something like:

```
{ "userId": 1, "id": 4, "title": "et porro tempora", "completed": true }
```

We have used `title` and `completed` all along. We have **never used** `userId` .

`userId` is a number. It does not tell you who the person is — it only tells you **which** person. The names live somewhere else:

```
https://jsonplaceholder.typicode.com/users
```

Open that address too. Find the property that holds the person's name, and the property you would compare against a task's `userId` .

Do not start typing until you can answer this out loud: *which number in the first list has to match which number in the second list?*

Why we will load more tasks

Check the `userId` of the fifteen tasks you have now. They all belong to the **same person**. A feature about *who owns what* is not worth much when everybody is the same somebody.

Loading **50** tasks gives us a useful set that belongs to **three** different people, which is enough to see the feature work. That is why step 1 below asks for 50.

CORE — everybody does this

1. Show who owns each task

Every task belongs to a person. Show that person's **name** in small grey text under the task title.

What it must do

- The page loads **50** tasks instead of 15.
- The page also asks the API for the **people**.
- Each row shows the name of the person whose `id` matches that task's `userId` .
- If no matching person is found, the row shows something sensible — **never** `undefined` .

How to approach it

1. Change the number in `API_URL` so the page loads 50 tasks. Check that the statistic cards follow by themselves, and be ready to say why nothing else in your code had to change.
2. You need a second address. Store it the same way you stored `API_URL` .

3. You need somewhere to keep the people once they arrive. Look at how `tasks` is declared and do the same thing.
4. Write an `async` function that asks for the people and fills that array. It is the same three steps as the tasks request: ask, wait for the answer, turn it into objects.
5. Write a function called `getUserName(userId)`. It receives **one number** and returns **one piece of text**. Inside it, loop through the people and compare.
6. Use it inside the template literal in `renderTasks()`.
7. Give the name a class and style it small and grey in `css/style.css`.

Hint: `getUserName` is a new *shape* of function for you. `setFilter` receives values but gives nothing back; `getVisibleTasks` gives something back but receives nothing. This one does both.

Hint: you can `return` from **inside** a `for...of` loop. The moment you `return`, the loop stops and the function is finished.

Hint: what should `getUserName` return if the loop finishes without finding anybody? Write that line **before** you test, not after.

Hint: think about **when** the people must arrive. If the rows are drawn before the names are here, every row will show your fallback text. Which keyword makes JavaScript wait?

Hint: the API sends the people as an array of objects, exactly like the tasks. Nothing about the loop is new — only what you compare.

CORE acceptance criteria

- ☐ 50 tasks load from the todos API, and the **Total Tasks** card says 50 by itself.
 - ☐ The people are loaded from the users API.
 - ☐ Every task row shows a person's name under its title.
 - ☐ The name is the one whose `id` matches that task's `userId`.
 - ☐ The word `undefined` never appears anywhere on the page.
 - ☐ The names are grey and smaller than the task title.
 - ☐ More than one person's name appears across the list.
 - ☐ The **All / Completed / Pending** buttons still work.
 - ☐ The search box still works.
 - ☐ The statistic cards and the progress sentence still show the right numbers.
 - ☐ In DevTools → **Network** → **Fetch/XHR**, you see **two API requests**: one for the todos and one for the users.
 - ☐ No red errors in the console while the app is working normally.
-

STRETCH — if CORE was comfortable

2. How busy is each person?

Now that you have both lists, you can work out something the API never sent you: **how many of the loaded tasks belong to each person.**

Add a small summary somewhere above the task list, for example:

```
Leanne Graham – 20 tasks
Ervin Howell – 20 tasks
Clementine Bauch – 10 tasks
```

What it must do

- One line per person, showing their name and how many of the loaded tasks are theirs.
- The numbers are **worked out from the data** — never typed in by hand.
- A person who owns none of the loaded tasks must not appear at all.
- Adding all the numbers up gives the number in the **Total Tasks** card.
- The summary describes **everything that was loaded**, so it must **not** change when the user clicks a filter or types in the search box.
- The lines are **information only**. Nothing here is clickable yet — that is the CHALLENGE.

Hint: you already know how to count with a loop. The new part is counting *separately for each person*, which means one loop running inside another one. Which list is the outer loop?

Hint: the "must not change when filtering" requirement is the interesting one. Look at where your existing functions are called from. One of them runs on every keystroke; another runs only when new data arrives. Choosing the wrong one is the mistake this task is built to catch.

3. Do not let a missing name break the app

The names are a **bonus**. The task list is the **product**.

Right now, if the users request fails, your whole `loadTasks()` gives up and the user sees the red error message — even though the tasks themselves were perfectly available.

Change that. If the people cannot be loaded but the tasks can:

- the task list must still appear and stay usable,
- the filters, the search and the statistics must all still work,
- the rows can show your fallback text instead of a name.

Hint: you already know the tool for this. The question is *where* to put it. At the moment there is one `try / catch` protecting everything. What happens if the people request gets its own?

STRETCH acceptance criteria

- ☐ A summary lists people and how many tasks each one has.
 - ☐ The counts are calculated from the loaded tasks, not written by hand.
 - ☐ Everybody who owns at least one loaded task is listed, and nobody else.
 - ☐ The counts add up to the number in the **Total Tasks** card.
 - ☐ Clicking **Completed** or **Pending** does **not** change the summary.
 - ☐ Typing in the search box does **not** change the summary.
 - ☐ The task list itself still reacts to the filters and the search exactly as before.
 - ☐ If the users request fails, the tasks still load and the app stays usable.
 - ☐ The red error message still appears when the **tasks** cannot be loaded.
 - ☐ No red errors in the console.
-

CHALLENGE — for students who want more

4. Click a person, see only their tasks

Turn the summary from the STRETCH into something the user can use.

Clicking a person shows **only that person's tasks**. An **All people** option brings everybody back.

What it must do

- Clicking a person narrows the task list to that person's tasks.
- The chosen person is visibly marked as chosen.
- An **All people** option restores the full list.
- It works **together** with the two things that already narrow the list. Choosing a person, then clicking **Pending**, then typing in the search box must leave only the tasks that satisfy **all three** at once — in any order the user does them.
- Changing the status filter does **not** clear the chosen person.
- Clearing the search box does **not** clear the chosen person.
- The statistic cards and the summary counts keep describing **all** loaded tasks, not the visible ones.
- When nothing matches all three, a message is shown instead of an empty area.

Strategic hints only

On state: the application already remembers two choices the user has made. A third one works the same way — and the place where those choices are turned into a list of rows is the place your third choice belongs too.

On the click: this is the genuinely new part. Until now, every element you attached a listener to was written by hand in `index.html` and existed before your JavaScript ran. These person lines do not — **your own code creates them**, and creates them again every time the summary is drawn.

Think carefully about the order of events. An element cannot be listened to before it exists, and replacing `innerHTML` throws the old elements away.

On getting the number back: when the listener finally runs, it needs to know *which* person was clicked. There is more than one honest way to arrange that. Find one you can explain.

CHALLENGE acceptance criteria

- ☐ Clicking a person in the summary shows only that person's tasks.
- ☐ The chosen person looks different from the others.
- ☐ An **All people** option brings all the tasks back.
- ☐ The status filter still works while a person is chosen.
- ☐ The search still works while a person is chosen.
- ☐ Person + status + search all apply at the same time, in any order.
- ☐ Changing the status filter does not clear the chosen person.
- ☐ Clearing the search box does not clear the chosen person.
- ☐ Choosing a person does **not** change the three statistic cards.
- ☐ Choosing a person does **not** change the summary counts.
- ☐ When nothing matches all three, a message such as "No tasks to show." appears.
- ☐ Reloading the page starts again with everybody's tasks.
- ☐ No red errors in the console.

Do not break what already works

Your new code is added to a working application. Check the old features before you hand in:

- ☐ Tasks still arrive from the API when the page opens.
- ☐ **All**, **Completed** and **Pending** still work, and the clicked button is still highlighted.
- ☐ The search box still filters by title while you type.

- ☐ The search still ignores capital letters (`DELECTUS` finds *delectus*).
- ☐ Clearing the search box shows the tasks again.
- ☐ Search and the status buttons still work together.
- ☐ The three statistic cards still show the right numbers.
- ☐ The progress sentence under the list is still correct.
- ☐ With the wifi off, the red error message still appears.
- ☐ No red errors in the console.

A feature that works is not finished if it broke something else. That check is part of the job.

How to hand in

1. Make sure both addresses are the real ones and the app works with the wifi on.
2. Keep your file structure exactly as it is:

```
index.html
css/style.css
js/app.js
```

1. Be ready to **explain how a number in one list found a name in the other list**.
-

Reminders

- Look at the data before you use it. Open both addresses in a browser tab first.
- A request that fails is not always fatal. Decide which data is essential and which is a bonus.
- `fetch` only fails when it cannot reach the server at all.
- Do not use `.filter()`, `.map()`, `.reduce()`, `.sort()` or `.find()` yet. Practise the loop.
- Keep the look in CSS and the decisions in JavaScript.